Methods Section Draft

Neural Operators for Density Evolution in Impulsive Dynamical Systems: Learning the Perron-Frobenius Operator Beyond the Lyapunov Horizon

[Authors]

February 16, 2026

Abstract

This document provides the detailed Methods section for a paper introducing neural operator architectures capable of learning density evolution in dynamical systems with impulsive (non-smooth) dynamics. We focus on one-dimensional hard-sphere gases as a paradigmatic example of chaotic systems where traditional physics-informed neural networks fail due to non-differentiable collision dynamics. Our approach bypasses the Lyapunov time horizon by learning the Perron-Frobenius operator directly from coarse-grained density trajectories.

Contents

1 Problem Formulation **2**
1.1 Physical System: 1D Hard-Sphere Gas on a Ring . . . 2
1.2 Dynamics: Event-Driven Molecular Dynamics . . . 2
1.3 The Lyapunov Time Horizon . . . 2
1.4 The Perron-Frobenius Operator . . . 3
1.5 Coarse-Grained Density Fields . . . 3

2 Neural Operator Architecture **3**
2.1 Fourier Neural Operator Framework . . . 3
2.2 Spectral Convolution Layer . . . 4
2.3 Full Layer Structure . . . 4
2.4 Architecture Hyperparameters . . . 4
2.5 Multi-Step Prediction . . . 5

3 Training Methodology **5**
3.1 Data Generation . . . 5
3.2 Dataset Construction . . . 6
3.3 Loss Function . . . 6
3.4 Optimization . . . 6

4 Evaluation Metrics **6**
4.1 Single-Step Accuracy . . . 6
4.2 Lyapunov Horizon Analysis . . . 7
4.3 Statistical Consistency . . . 7
4.4 Generalization Tests . . . 8

5 Baseline Methods **8**
5.1 Extended Dynamic Mode Decomposition (EDMD) 8
5.2 Ulam's Method 8
5.3 Recurrent Neural Networks 8
5.4 Convolutional Neural Networks 8

6 Implementation Details **9**
6.1 Software 9
6.2 Computational Resources 9
6.3 Reproducibility 9

7 Extension: Inverse Problem **9**
7.1 Problem Setup 9
7.2 Collision Graph Neural Network 10

8 Limitations and Future Work **10**
8.1 Current Limitations 10
8.2 Future Directions 10

1 Problem Formulation

1.1 Physical System: 1D Hard-Sphere Gas on a Ring

We consider a system of N hard spheres constrained to a one-dimensional circular track of length L. Each particle $i \in \{1, \ldots, N\}$ is characterized by:

- Position $x_i \in [0, L)$ with periodic boundary conditions
- Velocity $v_i \in \mathbb{R}$
- Diameter $d_i > 0$
- Mass $m_i > 0$

The system satisfies the *single-file diffusion* (SFD) constraint: particles cannot pass one another, preserving their ordering at all times. This is naturally enforced by the hard-sphere exclusion.

Definition 1 (System State)**.** *The microscopic state of the system is the vector $\boldsymbol{z} = (\boldsymbol{x}, \boldsymbol{v}) \in \Gamma$, where $\boldsymbol{x} = (x_1, \ldots, x_N)$ and $\boldsymbol{v} = (v_1, \ldots, v_N)$. The phase space $\Gamma \subset [0, L)^N \times \mathbb{R}^N$ is constrained by the non-overlap condition:*

$$|x_i - x_j|_L > \frac{d_i + d_j}{2}, \quad \forall i \neq j \tag{1}$$

where $|\cdot|_L$ denotes the distance on the circle of circumference L.

1.2 Dynamics: Event-Driven Molecular Dynamics

Between collisions, particles move freely:

$$\dot{x}_i = v_i, \quad \dot{v}_i = 0 \tag{2}$$

Collisions occur instantaneously when two adjacent particles i and j come into contact. The post-collision velocities are determined by conservation of momentum and kinetic energy:

$$v_i' = \frac{(m_i - m_j)v_i + 2m_j v_j}{m_i + m_j} \tag{3}$$

$$v_j' = \frac{(m_j - m_i)v_j + 2m_i v_i}{m_i + m_j} \tag{4}$$

Remark 1 (Integrability vs. Chaos)**.** *When all masses are equal ($m_i = m$ for all i), the collision rule simplifies to velocity exchange ($v_i' = v_j$, $v_j' = v_i$), making the system integrable. With unequal masses and $N \geq 3$, the system becomes chaotic with positive Lyapunov exponents [2].*

1.3 The Lyapunov Time Horizon

The maximum Lyapunov exponent $\lambda_{\max}$ quantifies the exponential divergence of nearby trajectories:

$$\|\delta \boldsymbol{z}(t)\| \sim \|\delta \boldsymbol{z}(0)\| e^{\lambda_{\max} t} \tag{5}$$

This imposes a fundamental limit on deterministic prediction. The *Lyapunov time* is defined as:

$$T_{\text{Lyap}} = \frac{1}{\lambda_{\max}} \ln\left(\frac{\Delta_{\text{macro}}}{\delta_0}\right) \tag{6}$$

where δ_0 is the initial uncertainty and Δ_{macro} is the macroscopic scale of the system. Beyond T_{Lyap}, pointwise trajectory prediction becomes impossible regardless of computational resources.

1.4 The Perron-Frobenius Operator

To bypass the Lyapunov barrier, we shift from tracking individual trajectories to tracking the evolution of probability densities over phase space.

Definition 2 (Perron-Frobenius Operator)**.** *Let $\Phi^t : \Gamma \to \Gamma$ denote the flow map of the dynamical system. The Perron-Frobenius (PF) operator $\mathcal{P}^t : L^1(\Gamma) \to L^1(\Gamma)$ acts on probability densities $\rho : \Gamma \to \mathbb{R}_{\geq 0}$ as:*

$$(\mathcal{P}^t \rho)(\boldsymbol{z}) = \int_\Gamma \delta\big(\boldsymbol{z} - \Phi^t(\boldsymbol{z}')\big)\rho(\boldsymbol{z}')\, d\boldsymbol{z}' \tag{7}$$

Proposition 1 (Linearity)**.** *The Perron-Frobenius operator is linear, even when the underlying dynamics Φ^t are highly nonlinear:*

$$\mathcal{P}^t(\alpha\rho_1 + \beta\rho_2) = \alpha\mathcal{P}^t\rho_1 + \beta\mathcal{P}^t\rho_2 \tag{8}$$

This linearity is the key insight enabling neural operator learning: we learn a linear map between infinite-dimensional function spaces, which neural operators are designed to approximate.

1.5 Coarse-Grained Density Fields

Working in the full $2N$-dimensional phase space is computationally prohibitive. We project to a one-dimensional *spatial density field* by marginalizing over velocities and particle labels:

$$\rho(x, t) = \sum_{i=1}^{N} \mathbb{E}\left[K_h(x - x_i(t))\right] \tag{9}$$

where K_h is a smoothing kernel with bandwidth h (we use a Gaussian kernel with $h = 2\max_i d_i$). This coarse-graining:

1. Reduces dimensionality from $2N$ to a fixed spatial resolution
2. Smooths the discontinuities inherent in hard-sphere dynamics
3. Produces the physically meaningful observable (particle density)

Our goal is to learn the *projected* PF operator:

$$\mathcal{G}_{\Delta t} : \rho(\cdot, t) \mapsto \rho(\cdot, t + \Delta t) \tag{10}$$

2 Neural Operator Architecture

2.1 Fourier Neural Operator Framework

We employ the Fourier Neural Operator (FNO) architecture [1], which learns mappings between infinite-dimensional function spaces. The FNO is particularly suited to our problem because:

- It naturally handles periodic boundary conditions (circular track)
- Spectral methods efficiently capture global correlations
- The architecture is resolution-invariant

Definition 3 (Fourier Neural Operator)**.** *The FNO maps an input function $\rho_{in} : [0, L] \to \mathbb{R}$ to an output function $\rho_{out} : [0, L] \to \mathbb{R}$ through the composition:*

$$\rho_{out} = Q \circ \sigma(\mathcal{W}_T + \mathcal{F}_T) \circ \cdots \circ \sigma(\mathcal{W}_1 + \mathcal{F}_1) \circ P(\rho_{in}) \tag{11}$$

where:

- *$P : \mathbb{R} \to \mathbb{R}^{d_h}$ lifts the input to a higher-dimensional channel space*
- *$\mathcal{F}_\ell$ are spectral convolution layers*
- *$\mathcal{W}_\ell$ are local linear transforms (pointwise)*
- *σ is a nonlinear activation (GeLU)*
- *$Q : \mathbb{R}^{d_h} \to \mathbb{R}$ projects back to the output space*

2.2 Spectral Convolution Layer

The spectral convolution $\mathcal{F}$ operates in Fourier space, enabling efficient global computation:

$$(\mathcal{F}u)(x) = \mathcal{F}^{-1}\left(R \cdot \mathcal{F}(u)\right)(x) \tag{12}$$

where $\mathcal{F}$ and $\mathcal{F}^{-1}$ denote the Fourier transform and its inverse, and $R \in \mathbb{C}^{d_h \times d_h \times k_{\max}}$ are learnable complex weights for each of the retained $k_{\max}$ Fourier modes.

In discrete form, for input $u \in \mathbb{R}^{d_h \times n}$ (channels $\times$ spatial points):

$$\hat{u} = \mathrm{FFT}(u) \in \mathbb{C}^{d_h \times (n/2+1)} \tag{13}$$

$$\hat{v}_k = \sum_{j=1}^{d_h} R_{jk} \hat{u}_{jk}, \quad k = 1, \ldots, k_{\max} \tag{14}$$

$$v = \mathrm{IFFT}(\hat{v}) \in \mathbb{R}^{d_h \times n} \tag{15}$$

Higher Fourier modes ($k > k_{\max}$) are truncated, acting as an implicit low-pass filter. This is appropriate for coarse-grained density fields where high-frequency microscopic details are already smoothed.

2.3 Full Layer Structure

Each Fourier layer combines spectral and local pathways:

$$u^{(\ell+1)} = \sigma\left(\mathcal{F}_\ell(u^{(\ell)}) + W_\ell u^{(\ell)} + b_\ell\right) \tag{16}$$

where $W_\ell \in \mathbb{R}^{d_h \times d_h}$ is a pointwise linear transform and $b_\ell \in \mathbb{R}^{d_h}$ is a bias. The local pathway captures features that are not well-represented in the truncated Fourier basis.

2.4 Architecture Hyperparameters

Based on preliminary experiments, we use:

Hyperparameter	Symbol	Value
Spatial resolution	n	128
Fourier modes retained	$k_{\max}$	32
Hidden channels	d_h	64
Number of Fourier layers	T	4
Activation function	σ	GeLU

Table 1: FNO architecture hyperparameters.

2.5 Multi-Step Prediction

For long-horizon prediction, we apply the learned operator autoregressively:

$$\hat{\rho}_{t+k\Delta t} = \underbrace{\mathcal{G}_{\Delta t} \circ \cdots \circ \mathcal{G}_{\Delta t}}_{k \text{ times}}(\rho_t) \tag{17}$$

Autoregressive rollout enables prediction far beyond T_{Lyap}, limited only by the accumulation of approximation errors.

3 Training Methodology

3.1 Data Generation

Training data is generated via event-driven molecular dynamics simulation.

Algorithm 1 Event-Driven Hard-Sphere Simulation

Require: Initial state $\boldsymbol{z}_0 = (\boldsymbol{x}_0, \boldsymbol{v}_0)$, duration T, save interval Δt_{save}

```
t ← 0
states ← [z_0]
while t < T do
    t_coll, (i, j) ← FindNextCollision(z)
    if t_coll > T then
        AdvanceParticles(T − t)
        break
    end if
    while next save time < t + t_coll do
        AdvanceParticles(next save time − t)
        states.append(z)
    end while
    AdvanceParticles(t_coll)
    (v_i, v_j) ← ResolveCollision(i, j)        ▷ Eqs. (3)–(4)
end while
return states
```

For each trajectory, we record:

- Snapshots of the density field $\rho(x, t_k)$ at intervals Δt_{save}
- Collision events $(t, i, j, \Delta p)$ for auxiliary analysis

- System parameters $(N, \{m_i\}, \{d_i\}, L)$ as metadata

3.2 Dataset Construction

We generate a diverse dataset spanning the parameter space:

Parameter	**Range**	**Sampling**
Number of particles N	$[3, 15]$	Uniform integer
Track length L	1000 (fixed)	—
Diameter d_i	$[5, 25]$	Uniform
Mass m_i	$\propto d_i^3$	Equal density
Initial speed $\lvert v_i \rvert$	$[20, 80]$	Uniform
Initial direction	± 1	Uniform
Simulation time	500	Fixed
Save interval Δt	0.1	Fixed

Table 2: Dataset generation parameters.

Each trajectory yields approximately 5000 input-output pairs $(\rho_t, \rho_{t+\Delta t})$.

Dataset size: We generate $M = 1000$ trajectories, yielding $\sim 5 \times 10^6$ training samples after windowing.

3.3 Loss Function

We minimize the mean squared error between predicted and true density fields:

$$\mathcal{L}(\theta) = \frac{1}{M} \sum_{m=1}^{M} \frac{1}{T_m} \sum_{t=1}^{T_m} \left\| \mathcal{G}_{\Delta t}^{(\theta)}(\rho_t^{(m)}) - \rho_{t+\Delta t}^{(m)} \right\|_2^2 \tag{18}$$

where θ denotes the network parameters.

Auxiliary Losses (Optional): To encourage physical consistency, we optionally add:

- **Mass conservation:** $\mathcal{L}_{\text{mass}} = \left(\int_0^L \hat{\rho}(x) dx - \int_0^L \rho(x) dx \right)^2$
- **Positivity:** $\mathcal{L}_{\text{pos}} = \| \min(\hat{\rho}, 0) \|_2^2$

3.4 Optimization

Training is performed on a single NVIDIA A100 GPU, requiring approximately 8 hours for the full dataset.

4 Evaluation Metrics

4.1 Single-Step Accuracy

For immediate prediction quality, we report:

$$\text{MSE}_1 = \mathbb{E} \left[\| \mathcal{G}_{\Delta t}(\rho_t) - \rho_{t+\Delta t} \|_2^2 \right] \tag{19}$$

Setting	Value
Optimizer	AdamW
Learning rate	10^{-3} (initial)
Learning rate schedule	Cosine annealing
Weight decay	10^{-4}
Batch size	64
Epochs	200
Gradient clipping	$\|\nabla\|_2 \leq 1.0$

Table 3: Training configuration.

and the relative L^2 error:

$$\epsilon_{\text{rel}} = \mathbb{E}\left[\frac{\|\mathcal{G}_{\Delta t}(\rho_t) - \rho_{t+\Delta t}\|_2}{\|\rho_{t+\Delta t}\|_2}\right] \tag{20}$$

4.2 Lyapunov Horizon Analysis

The key contribution of this work is predicting *beyond* the Lyapunov time. We evaluate:

Definition 4 (Prediction Horizon)**.** *The prediction horizon $T_{pred}(\epsilon)$ is the time at which the relative error first exceeds threshold ϵ:*

$$T_{pred}(\epsilon) = \inf\left\{t : \frac{\|\hat{\rho}_t - \rho_t\|_2}{\|\rho_t\|_2} > \epsilon\right\} \tag{21}$$

We report T_{pred} normalized by the Lyapunov time:

$$\tau_{\text{pred}} = \frac{T_{\text{pred}}}{T_{\text{Lyap}}} \tag{22}$$

A value $\tau_{\text{pred}} > 1$ indicates successful prediction beyond the deterministic horizon.

4.3 Statistical Consistency

Even when pointwise density prediction degrades, the predicted density should maintain correct statistical properties. We evaluate:

Invariant Measure: For long-time predictions, the time-averaged density should converge to the microcanonical equilibrium:

$$\bar{\rho}_{\text{pred}} = \frac{1}{T}\int_0^T \hat{\rho}(\cdot, t)\, dt \approx \rho_{\text{eq}} \tag{23}$$

Autocorrelation Structure: We compare the temporal autocorrelation function:

$$C(\tau) = \langle \rho(x,t)\rho(x,t+\tau)\rangle - \langle\rho\rangle^2 \tag{24}$$

between predicted and true trajectories.

4.4 Generalization Tests

We test generalization across:

1. **Out-of-distribution N:** Train on $N \in [3, 10]$, test on $N \in [11, 15]$
2. **Different mass distributions:** Train on $m \propto d^3$, test on uniform masses
3. **Higher resolution:** Train at $n = 128$, test at $n = 256$

5 Baseline Methods

We compare against:

5.1 Extended Dynamic Mode Decomposition (EDMD)

EDMD approximates the Koopman operator using a dictionary of observables [3]. We use:

- Fourier basis functions up to mode $k = 32$
- Regularized least-squares fitting

5.2 Ulam's Method

Ulam's method discretizes the PF operator by partitioning phase space into cells and computing transition probabilities [4]. We use 100 spatial bins.

5.3 Recurrent Neural Networks

We implement an LSTM baseline:

- Input: Flattened density field
- Hidden dimension: 256
- Layers: 2
- Output: Next density field

5.4 Convolutional Neural Networks

A U-Net style architecture:

- Encoder: 4 convolutional blocks with downsampling
- Decoder: 4 transposed convolutional blocks
- Skip connections between encoder and decoder

6 Implementation Details

6.1 Software

The codebase is implemented in:

- **Physics simulation:** Python with NumPy (event-driven MD)
- **Neural operators:** PyTorch 2.0 with custom FNO layers
- **Training infrastructure:** PyTorch Lightning
- **Experiment tracking:** Weights & Biases

Code and trained models will be released at: `github.com/[redacted]/impulsive-neural-operators`

6.2 Computational Resources

Component	Resource
Data generation	32-core CPU cluster, 24 hours
FNO training	1× NVIDIA A100 (40GB), 8 hours
Baseline training	1× NVIDIA V100 (32GB), 4–12 hours each
Evaluation	1× NVIDIA A100, 2 hours

Table 4: Computational resources used.

6.3 Reproducibility

All random seeds are fixed:

- Data generation: seed = 42
- Train/validation/test split: 80/10/10
- Model initialization: seed = 0

Confidence intervals are computed over 5 independent training runs.

7 Extension: Inverse Problem

As a secondary contribution, we address the *inverse problem*: inferring particle masses and diameters from collision statistics.

7.1 Problem Setup

Given a sequence of collision events:

$$\mathcal{C} = \{(t_k, i_k, j_k, \Delta p_k)\}_{k=1}^{K} \tag{25}$$

where t_k is the collision time, (i_k, j_k) are the colliding particles, and Δp_k is the momentum transfer magnitude, we seek to recover $\{m_i, d_i\}_{i=1}^{N}$.

7.2 Collision Graph Neural Network

We propose a graph neural network operating on the collision graph:

- **Nodes:** Particles, with learnable embeddings
- **Edges:** Collision events between particle pairs
- **Edge features:** $(t_k, \Delta p_k, x_k)$
- **Message passing:** Aggregate collision information to each particle
- **Output heads:** Predict (m_i, d_i) for each particle

This architecture handles the non-differentiability challenge by operating on the discrete collision graph rather than continuous trajectories.

8 Limitations and Future Work

8.1 Current Limitations

1. **Coarse-graining dependence:** The choice of kernel bandwidth h affects what dynamics are captured. Very small h reintroduces discontinuities; very large h over-smooths.
2. **Fixed Δt:** The current model is trained for a specific time step. Adaptive time-stepping would require architecture modifications.
3. **Homogeneous track:** We assume a uniform circular track. Extensions to heterogeneous media (e.g., varying diffusivity) are left for future work.

8.2 Future Directions

1. **Biological applications:** Apply to polyribosome dynamics on circular mRNA, where ribosomes act as hard spheres with time-varying masses (due to nascent polypeptide chains).
2. **Higher dimensions:** Extend to 2D and 3D hard-sphere gases, relevant to granular flows and colloidal suspensions.
3. **Stochastic extensions:** Incorporate thermal noise to bridge deterministic chaos and Brownian dynamics.

References

[1] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar. Fourier Neural Operator for Parametric Partial Differential Equations. In *International Conference on Learning Representations*, 2021.

[2] Y. G. Sinai. Dynamical systems with elastic reflections. *Russian Mathematical Surveys*, 25(2):137–189, 1970.

[3] M. O. Williams, I. G. Kevrekidis, and C. W. Rowley. A data-driven approximation of the Koopman operator: Extending dynamic mode decomposition. *Journal of Nonlinear Science*, 25(6):1307–1346, 2015.

[4] S. M. Ulam. *A Collection of Mathematical Problems*. Interscience Publishers, 1960.